

Aula 12 — Quando um colaborador começa a variar

Como evoluir uma colaboração quando o objeto que realiza o trabalho pode variar?

DO PROJETO QUE JÁ FUNCIONA

Sabemos acrescentar regras

- Aula 09: `Pedido` protege o fechamento.
- Aula 10: localiza linhas; o item protege sua quantidade.
- Aula 11: o raciocínio funciona em outro domínio.

Agora chegou uma responsabilidade que ainda não existia.

NOVO REQUISITO

O pedido precisa de uma forma de entrega

Por enquanto, só existe **entrega normal**.

Custo fixo: **R\$ 10**.

Quem deveria conhecer essa regra?

UMA RESPONSABILIDADE PRÓPRIA

Um objeto pode realizar esse trabalho

```
public class EntregaNormal {  
    public double calcularCusto(Pedido pedido) {  
        return 10.0;  
    }  
}
```

Recebe o pedido como contexto. Nesta regra fixa, não precisa consultar seus dados.

Pedido mantém a referência necessária

```
private EntregaNormal entrega;  
  
public Pedido(EntregaNormal entrega) {  
    this.entrega = entrega;  
    itens = new ArrayList<>();  
    fechado = false;  
}
```

Os campos `itens` e `fechado` continuam na classe.

SOLICITAR O COMPORTAMENTO

Pedido coordena o cálculo da entrega

```
public double calcularCustoEntrega() {  
    return entrega.calcularCusto(this);  
}
```

Pedido precisa saber que a regra devolve 10.0 ?

Java em foco — o próprio pedido

```
return entrega.calcularCusto(this);
```

`this` referencia o `Pedido` que executa o método.

O colaborador recebe esse mesmo pedido como argumento.

Nenhuma cópia é criada.

A NOVA DEPENDÊNCIA

Uma entrega, uma colaboração

Pedido



EntregaNormal

```
private EntregaNormal entrega;
```

Visão simplificada: a referência mantida no campo permite alcançar a entrega.

ATIVIDADE

EXPLORE ANTES DE EVOLUIR

A distribuição faz sentido?

1. Quem conhece a fórmula do frete?
2. Se ela mudar para R\$ 12, onde esperamos alterar código?
3. Essa solução é adequada enquanto existe só entrega normal?

O QUE JÁ CONSEGUIMOS PRESERVAR

A regra pertence à entrega

`EntregaNormal` conhece o custo e pode mudar sua regra.

`Pedido` continua solicitando o mesmo cálculo.

A solução é adequada ao requisito atual.

Dois resultados, duas responsabilidades

```
Produto teclado = new Produto("Teclado", 150.0);  
Produto mouse = new Produto("Mouse", 80.0);  
Pedido pedido = new Pedido(new EntregaNormal());  
pedido.adicionarItem(teclado, 2);  
pedido.adicionarItem(mouse, 1);  
System.out.println(pedido.calcularTotal());  
System.out.println(pedido.calcularCustoEntrega());
```

Quais valores serão impressos? De onde vem cada um?

O SISTEMA CONTINUA FUNCIONANDO

Itens e entrega têm cálculos separados

380.0

10.0

`calcularTotal()` continua somando subtotais dos itens.

`calcularCustoEntrega()` solicita a nova colaboração.

O REQUISITO EVOLUIU

Alguns pedidos precisam de entrega expressa

```
public class EntregaExpressa {  
    public double calcularCusto(Pedido pedido) {  
        return 25.0;  
    }  
}
```

Também calcula o custo da entrega. Usa outra regra.

ATIVIDADE

UMA PREVISÃO DE COMPILAÇÃO

Podemos fornecer a nova entrega?

```
// Pedido recebe EntregaNormal no construtor.  
EntregaExpressa expressa = new EntregaExpressa();  
Pedido urgente = new Pedido(expressa);
```

O compilador aceita essa chamada? Por quê?

ONDE ESTÁ O LIMITE?

O parâmetro pede uma classe concreta

```
public Pedido(EntregaNormal entrega)
```

O argumento é `EntregaExpressa` .

No código mostrado, esse tipo não é aceito como `EntregaNormal` .

Ter `calcularCusto(Pedido)` nas duas classes não basta.

O QUE MUDOU PARA O PEDIDO?

Ainda precisamos do mesmo comportamento

Pedido continua precisando **calcular o custo da entrega**.

Agora existem duas classes capazes de realizar esse trabalho.

A responsabilidade mudou ou mudou quem pode realizá-la?

Trocar objetos não é sempre trocar classes

Na missão:

```
Sensor temperatura = new Sensor("Temperatura", "°C");  
Sensor umidade = new Sensor("Umidade", "%");
```

Dois objetos da mesma classe.

Agora: `EntregaNormal` e `EntregaExpressa` são classes diferentes.

ATIVIDADE

INVESTIGUE COM O REPERTÓRIO ATUAL

Como manter as duas alternativas?

Proponha uma solução para pedidos normais e expressos coexistirem.

Que referências **Pedido** manteria?

Como saberia qual comportamento solicitar?

UMA PROPOSTA POSSÍVEL

Duas referências concretas

```
private EntregaNormal entregaNormal;  
private EntregaExpressa entregaExpressa;
```

Ainda precisamos indicar qual forma pertence àquele pedido.

Quem passa a conhecer as duas classes?

OUTRA PROPOSTA POSSÍVEL

Guardar a escolha no pedido

```
private int tipoEntrega;
```

Nesta hipótese: 1 significa normal, 2 significa expressa.

Um texto como "NORMAL" também poderia representar a escolha.

Quem precisa conhecer o significado dessas alternativas?

Pedido decide a quem solicitar

```
public double calcularCustoEntrega() {  
    if (tipoEntrega == 1) {  
        return entregaNormal.calcularCusto(this);  
    } else {  
        return entregaExpressa.calcularCusto(this);  
    }  
}
```

Hipótese: código válido e dois colaboradores já disponíveis.

Que conhecimento chegou a Pedido?

Proposta	Conhecimento necessário no pedido
campos e seleção	classes concretas e escolha entre elas
fórmulas no pedido	alternativas e regras de cada cálculo

O pedido precisa conhecer os nomes das alternativas para continuar coordenando?

Uma condição pode proteger uma responsabilidade

```
if (!fechado) {  
    // Pedido autoriza uma edição de sua estrutura.  
}
```

Isso continua fazendo sentido.

Agora investigamos quanto o pedido precisa conhecer sobre **quem calcula a entrega**.

O CONJUNTO DE ALTERNATIVAS CRESCEU

O cliente também pode retirar no local

```
public class RetiradaLocal {  
    public double calcularCusto(Pedido pedido) {  
        return 0.0;  
    }  
}
```

Mais uma forma concreta de realizar a mesma responsabilidade.

ATIVIDADE

PREVEJA O IMPACTO

Criar a classe basta?

Na proposta com campos concretos e seleção em `Pedido` :

1. criar `RetiradaLocal.java` já integra a alternativa?
2. onde precisamos adaptar referências, preparação e seleção?
3. a responsabilidade de `Pedido` mudou de novo?

O QUE A TERCEIRA OPÇÃO REVELOU

A coordenação continua; as alternativas crescem

Só a classe nova não basta naquela proposta.

Pedido precisa ser adaptado para receber e seleccionar a retirada.

O cliente também precisa solicitar a nova opção.

A tarefa continua sendo solicitar o custo da entrega.

SEPARE O ESTÁVEL DO VARIÁVEL

O que permanece? O que varia?

Permanece	Varia
solicitar custo para um pedido	qual objeto responde
receber um resultado <code>double</code>	qual regra ele realiza
coordenação do pedido	classe concreta do colaborador

`EntregaNormal` · `EntregaExpressa` · `RetiradaLocal`

CONCEITO-CHAVE

AGORA PODEMOS NOMEAR O FENÔMENO

Conceito-chave — ponto de variação

Uma parte do sistema em que esperamos alternativas de comportamento.

Aqui: **calcular o custo da entrega.**

Normal, expressa e retirada são formas concretas de realizar essa responsabilidade.

A NECESSIDADE DECLARADA

O que esse campo está dizendo?

```
private EntregaNormal entrega;
```

“Preciso de um objeto `EntregaNormal` .”

Essa frase descreve tudo de que o pedido realmente precisa?

OBSERVE O USO

A necessidade pode ser descrita pelo trabalho

```
return entrega.calcularCusto(this);
```

“Preciso de algo capaz de calcular o custo da entrega para este pedido.”

Ainda falta representar isso em Java.

CONCEITO-CHAVE

DAR NOME À DEPENDÊNCIA OBSERVADA

Conceito-chave — acoplamento

Uma classe está acoplada a outra quando depende dela para realizar seu trabalho.

O campo e o construtor tornam `Pedido` dependente de `EntregaNormal`.

Essa dependência era adequada enquanto havia só essa alternativa.

ARMADILHA

UM CUIDADO COM O DIAGNÓSTICO

O que devemos investigar?

Dependências permitem colaboração. Condições podem expressar regras adequadas.

O critério desta aula:

Pedido precisa conhecer cada implementação concreta para continuar solicitando o mesmo trabalho?

ALGUÉM AINDA TERÁ DE ESCOLHER

E se Main criar a alternativa?

`Main` já monta os cenários e pode conhecer a opção escolhida.

Mas ainda precisa fornecer esse objeto a `Pedido`.

O campo `EntregaNormal` passaria a aceitar as outras classes só por mudar onde escolhemos?

UMA SEGUNDA PREOCUPAÇÃO

Vamos mudar uma colaboração que funciona

Mesmo que a entrega expressa passe a funcionar...

Como saber se preservamos o total, as edições permitidas e o fechamento?

Que verificações do Projeto 1 precisamos repetir?

Registre os resultados esperados

```
// teclado: 150.0; mouse: 80.0; pedido com entrega normal  
pedido.adicionarItem(teclado, 2);  
pedido.adicionarItem(mouse, 1);  
System.out.println(pedido.calcularTotal());  
pedido.removerItem(mouse);  
System.out.println(pedido.calcularTotal());  
pedido.alterarQuantidade(teclado, 3);  
System.out.println(pedido.calcularTotal());
```

Quais três valores são esperados?

CONTINUE O MESMO CENÁRIO

E depois de fechar?

```
pedido.fechar();  
pedido.adicionarItem(mouse, 1);  
pedido.removerItem(teclado);  
pedido.alterarQuantidade(teclado, 1);  
  
System.out.println(pedido.calcularTotal());  
System.out.println(pedido.calcularCustoEntrega());
```

Quais valores devem continuar corretos?

EVIDÊNCIAS QUE PODEMOS REPETIR

O comportamento esperado está explícito

```
380.0 → 300.0 → 450.0 → 450.0  
Entrega normal: 10.0
```

Também precisamos verificar zero, negativo e inclusão repetida.

Repetir e automatizar essas verificações prepara a próxima aula.

A mesma fronteira em um sistema de lembretes

Um lembrete solicita envio e conta avisos.

1. evoluir o canal de e-mail existente;
2. permitir também SMS;
3. acrescentar um painel local.

Onde o código precisa mudar a cada nova alternativa?

SÍNTESE

DA UNIDADE 01 À UNIDADE 02

Até aqui, quem realiza cada responsabilidade

- Unidade 01: objetos colaboram.
- Aula 12: o colaborador pode ser de outra classe.
- Queremos depender do trabalho necessário sem espalhar as alternativas.
- Antes de mudar a estrutura, precisamos verificar o que já funciona.

O PROBLEMA CONTINUA ABERTO

Pedido precisa de...

“Algo que calcule o custo da entrega.”

Alternativas: `EntregaNormal`, `EntregaExpressa`, `RetiradaLocal`.

Como representar essa necessidade em Java sem dizer qual classe concreta fará o trabalho?